

Subagent coordination: tokens, time and task completion

Cristhian Egoavil / 13 September 2026 / v1.0

Abstract

We review nine arXiv papers first submitted in 2026 and execute a small comparison of sequential, parallel, hierarchical and reflexive subagent management, with a single-agent baseline and a skill-loading ablation. Twelve synthetic tasks, two repetitions and six conditions produce 144 live runs and 416 model calls using GPT-5.6 Luna with low reasoning effort. The single agent completes 20/24 strict tasks with 1 143 tokens and 4.72 seconds per run on average; the four team configurations complete 19–20/24. A post-hoc SQL-wrapper sensitivity changes total completions from 120/144 to 142/144 without changing query text. Loading all skills increases hierarchical mean tokens by 94.8%. These exploratory results quantify overhead on short, fully supplied tasks and do not establish a universal best architecture or long-running autonomy.

Keywords: multi-agent systems; orchestration; agent skills; token efficiency; reproducible evaluation.

1 Introduction

A personal assistant can delegate work to specialized agents, but each delegation consumes resources before it produces useful work. Instructions must be prepared, context transferred, responses awaited and results integrated. A team can finish independent work sooner; it can also repeat an easy calculation several times without improving the answer. The relevant engineering question is therefore when coordination earns its overhead.

This study investigates how to choose a subagent architecture by balancing task completion, token consumption and elapsed time. It compares four bounded coordination patterns with an undivided solver and separately tests loading a complete skill library. The contribution is a transparent, small empirical comparison: fixed tasks, actual model calls, deterministic evaluation, per-call resource records and bilingual editable artifacts. It does not introduce a new trained model or establish a production architecture benchmark.

We distinguish three quantities that are often conflated. Task success concerns the delivered result. Total tokens measure the work processed across all calls, including repeated context. Elapsed time measures what a user waits through, allowing calls to overlap. A shorter context in one worker does not, by itself, imply fewer total tokens or lower RAM usage in the host application.

2 Research published in 2026

The review includes nine papers first submitted to arXiv between February and September 2026. Submission dates were checked separately from revision dates; exact consulted versions appear in the references. Selection targeted coordination, context transfer, skill assignment and measurable efficiency. This is a focused technical review, not an exhaustive systematic review. Other studies'

scores are not combined with the experiment below.

2.1 Execution topology and authority

Kulkarni and Kulkarni compare sequential, parallel, supervisor–worker and reflexive workflows for financial document extraction. Their descriptions motivate the four pattern families studied here. The domain and January 2025 model/pricing setup constrain transfer to a personal assistant; we do not reproduce their configurations or reported performance.¹

DESBench examines centralized, hierarchical, heterarchical and holonic coordination in industrial scheduling. Its implemented heterarchy uses mediated bidding. It expands the design question from graph shape to who may allocate work, reject commitments and resolve conflicts; its industrial simulator is a different evaluation environment.²

ParaGUI dispatches concurrent GUI workers in rounds and integrates their summaries. It provides evidence on decomposable interactive workloads, while identifying worker execution failures as a bottleneck. Its visual-history controls and multi-desktop setting make it unsuitable as a direct numerical baseline for this text-only experiment.³

2.2 Information transfer and evaluation

OrchBench represents dependencies and cross-agent transfers, then simulates quality, makespan and token cost. Its retention mechanism makes information loss a coordination variable. Our benchmark instead executes every worker against the API; simulated worker costs or outcomes are not substituted for observed usage or answers.⁴

CRAFT gives agents complementary partial views of a construction problem. It distinguishes individual reasoning from effective communication and describes repeated unproductive corrections. It motivates preserving source evidence and assessing the final result. Because every worker here receives the original task, we do not test its partial-observation problem.⁵

2.3 Skills and scoped workers

Subagents vs Agent Skills distinguishes procedural packages with explicit input/output interfaces from loosely structured skill text. Its main evaluation uses a selected 64-task subset and reports lower peak context with greater total token consumption. This supports testing payloads separately from topology.⁶

EvoAgent combines hierarchical delegation with keyword, embedding and model-based skill matching. Its domain-specific evaluation uses model judges; those scores do not establish exact execution reliability. We use its separation of retrieval and delegation as a reference, without implementing autonomous learning or reproducing its retrieval cascade.⁷

Swarm Skills packages roles, workflows, limits and dependencies as reusable coordination assets. Its qualitative case study leaves broad conformance testing unresolved. We treat portable

¹Kulkarni and Kulkarni, arXiv:2603.22651v1, 2026.

²Wang et al., arXiv:2605.13172v1, 2026.

³Yu et al., arXiv:2607.22689v1, 2026.

⁴Ren et al., arXiv:2607.25656v1, 2026.

⁵Nath, VanderHoeven and Krishnaswamy, arXiv:2603.25268v2, 2026.

⁶Piriyakulkij et al., arXiv:2609.09233v1, 2026.

⁷A. Zhang et al., arXiv:2604.20133v3, 2026.

workflow descriptions as a design proposal, not evidence that arbitrary multi-agent frameworks can interchange them without validation.⁸

Xu and Yan survey progressive disclosure: discover metadata, load chosen skill instructions, then retrieve supporting resources. This distinguishes a catalog from the content injected into each context. As a survey, it informs the loading policy but supplies no independent performance result for our architectures.⁹

3 Implemented architectures

All nodes call the same model through independent, stateless requests. A role is implemented by instructions and an explicit handoff; there is no separate trained specialist or persistent agent process. The four multi-agent conditions change the flow of information and decisions. They are deliberately small implementations of pattern families, not replicas of any cited framework.

Condition	Control flow	Calls
Single	One solver receives the complete request.	1
Sequential	Extract evidence; solve; finalize, with a rolling handoff.	3
Parallel	Two fixed complementary specialists run concurrently; a merger integrates.	3
Hierarchical	A supervisor selects 1–2 assignments; workers run; the supervisor integrates.	3–4
Reflexive	Propose; critique; revise if rejected. At most two critique/revision rounds.	2–5

Table 1: Primary comparison. The additional all-skills condition uses the same hierarchical control flow. A call means one complete API request.

Sequential. The first worker extracts facts and constraints. The second computes a candidate, and the final node reconciles it with the original request. Each stage receives the original input and only the preceding handoff, avoiding an unlimited transcript. This offers predictable sequencing but places all three model calls on the critical path.

Parallel. One worker focuses on data and principal calculations; the other on exceptions, scope and dependencies. Both receive the complete task. Their fixed assignments require no planning call, and the merger sees both outputs. This tests complementary parallel analysis, not an ensemble vote or parallel external actions.

Hierarchical. The first call outputs a JSON list of one or two concrete assignments. Workers receive their assignment and the original task; the final call receives the plan and findings. An invalid plan is a failure. The supervisor cannot create grandchildren, and this condition always delegates: the option of solving directly is represented by the separate baseline.

Reflexive. A proposer supplies a candidate. An independent critic either approves it with no issues or provides corrections. Revisions receive the candidate and critique. There are at most two feedback rounds, and the last candidate is evaluated even if the final allowed revision has no subsequent approval. The critic never sees reference answers or test results.

The primary conditions all receive a ten-entry skill metadata catalog and one relevant procedural

⁸X. Zhang et al., arXiv:2605.10052v2, 2026.

⁹Xu and Yan, arXiv:2602.12430v4, 2026.

skill. A separate hierarchical condition loads all ten full skill bodies at every node. Selection uses the known task-family label, not an inference call. This ablation isolates the payload policy; it does not establish that an assistant can always choose the correct skill from natural language.

4 Experimental method

4.1 Tasks and objective scoring

The evaluation contains twelve distinct, author-designed instances. They are synthetic and self-contained: task data, schema and business rules are supplied, while expected answers and database fixtures remain evaluator-side. No web access, filesystem mutation or terminal tool is exposed to the evaluated model. SQL is executed only in temporary in-memory databases with a read-only authorizer and a statement-work limit.

IDs	Family	Requirements tested
T01–02	Ledger	Resolve revisions before filtering; aggregate signed integer cents by project.
T03–04	Calendar	Earliest common interval for three participants, fixed UTC offsets and half-open busy intervals.
T05–06	Dependencies	Earliest starts and lexicographically resolved critical path across eleven DAG nodes.
T07–08	Selection	Select four of twelve items under budget, coverage, prerequisite and incompatibility constraints.
T09–10	Memory	Reduce 24 scoped events; preserve provenance, local overrides and unconfirmed candidates.
T11–12	SQL	Aggregate without multiplied joins; select latest attempts with ties; retain empty entities.

Table 2: Two instances per family. Each SQL instance has three distinct database fixtures; passing all three counts as one completed task.

Ten tasks require an exact structured answer. Ground truth uses event reducers, interval search, dynamic programming or exhaustive enumeration. The two SQL tasks require one query that passes all three fixtures. The primary output schema is an object containing only **answer**; its value must match the task-specified structure. Incorrect shapes fail the primary metric even when a person could interpret them. SQL fixture pass fractions are retained as secondary diagnostics.

Seven offline harness tests cover valid references, plausible wrong answers, SQL join multiplication and timestamp ties, read-only enforcement, concurrency, incomplete responses and token accounting. These tests validate measurement logic and are not counted as agent successes. A separate six-condition development pilot used a different ledger instance; its evidence is preserved but excluded from all performance tables.

4.2 Execution controls

Each task is run twice under six conditions: 144 evaluation runs, including 120 for the primary five-condition comparison and 24 for the skill ablation. Task/repetition blocks and condition order within blocks are shuffled using seed 20260913. Only one run executes at a time, with at most two simultaneous worker calls inside an eligible workflow. The local protocol and source hashes were fixed before evaluation; there was no external preregistration.

Every request uses `gpt-5.6-luna`, reasoning `low`, Responses API, JSON-object mode, non-streaming output and `store:false`. The per-call output cap is 1,536 tokens, including reasoning. Temperature and a model seed are not set. The model identifier and effort are supported by the official API documentation, which is implementation guidance outside the nine-paper corpus.¹⁰

The common ceiling is five calls per run. No new call starts after 240 seconds; HTTP connection/read timeouts are 10/60 seconds. This is a call-start deadline rather than a strict wall-time kill. Transport retries and application response caching are disabled. Provider input caching remains possible and is recorded. Failures are retained without selective reruns. Prompts, tasks, caps and the primary grader were not tuned on evaluation performance.

These controls equalize model, data access and maximum per-call output, not consumed compute. A one-call solver and a multi-call workflow naturally spend different amounts. The comparison therefore evaluates complete role-and-orchestration configurations; it cannot isolate topology from the effect of extra inference or role wording.

4.3 Metrics and uncertainty

For configuration a , let s_{ai} be whole-task success, t_{ai} the sum of provider-reported input and output tokens, and w_{ai} elapsed run time. With $N = 24$ runs per configuration, the principal measures are

$$S_a = \frac{1}{N} \sum_i s_{ai}, \quad \bar{T}_a = \frac{1}{N} \sum_i t_{ai}, \quad \bar{W}_a = \frac{1}{N} \sum_i w_{ai}. \quad (1)$$

Resource efficiency is $10^5 \sum_i s_{ai} / \sum_i t_{ai}$ successes per 100,000 tokens and $60 \sum_i s_{ai} / \sum_i w_{ai}$ successes per accumulated minute. Both include failed attempts. The latter describes serial-workload efficiency, not server throughput under saturation.

Tokens include supervisors, critics, repeated input and output reasoning. Cached input and reasoning counts are already subsets of the total and are not added twice. Wall time includes orchestration, network waiting and trace writes, excluding the final evaluator. The sum of API durations is also stored but is not used as elapsed latency when requests overlap. Missing usage would make token totals lower bounds rather than estimated replacements.

We use 10,000 paired cluster-bootstrap samples with seed 20260913: resample the twelve task ids and retain both repetitions and all conditions for each selected id. Percentile 95% intervals describe success, mean tokens and mean time; the skill comparison uses paired differences. The independent unit is the task instance, not every repeated run. Small-sample intervals are exploratory, and the synthetic task distribution cannot represent all assistant work.

5 Results

All 144 planned evaluation runs completed. The trace audit reconciles 416 model calls and 568 283 provider-reported tokens, including the 24-run skill ablation. There were 0 infrastructure or protocol execution errors; usage accounting is complete in every run. The six development-pilot runs are excluded. The preserved manifest and final hashes match, and the recorded order matches the frozen schedule.

Evaluation ran on 13 September 2026 from 13:09:45 to 13:33:34 UTC. This is an observed same-day API experiment, not a latency forecast for other accounts or deployments.

¹⁰OpenAI, GPT-5.6 Luna documentation, accessed 13 September 2026.

Condition	Strict	SQL norm.	Tokens/run	Mean s	Median s
Single agent	20/24	23/24	1 143	4.72	3.39
Sequential	20/24	24/24	3 410	13.72	11.77
Parallel	19/24	23/24	3 520	8.14	6.04
Hierarchical	20/24	24/24	4 373	11.19	10.01
Reflexive	20/24	24/24	2 714	9.77	6.26
Hierarchical + all skills	21/24	24/24	8 518	11.93	10.88

Table 3: Observed results. Strict is the frozen primary score. SQL norm. is the explicitly post-hoc wrapper sensitivity; query text is unchanged. The separated last row is the skill-loading ablation.

5.1 Format compliance and solution quality

During evaluation, visible SQL answers exposed a distinction between a correct-looking query and the requested nested result shape. The primary grader was left unchanged. An explicitly post-hoc sensitivity wraps an existing raw string under answer as an object with a sql field, then executes the identical query against the same three fixtures. No SQL text, numeric answer, prompt or trial is repaired or repeated.

This normalization applies to 22 recorded outputs. It changes whole-task completions from 120/144 to 142/144 across the six conditions. The strict column measures delivered schema compliance as well as correctness; the normalized column diagnoses how much of the observed failure is attributable to that wrapper. It is not a second prespecified endpoint or proof that every generated query is correct outside these fixtures.

The two remaining failures are T05, repetition 2, under the single and parallel conditions. Both return a correct makespan of 38 but start H at 10 instead of 18 and K at 21 instead of 29. H depends on C finishing, and K depends on H finishing. This is a dependency-propagation error, not a JSON-wrapper error; checking only the final makespan would miss it.

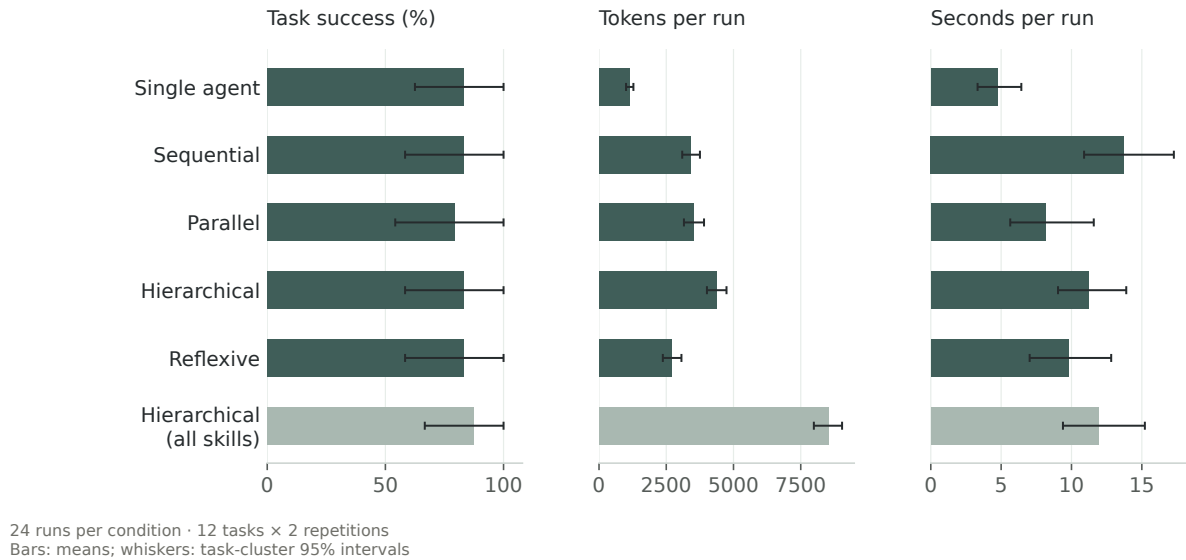


Figure 1: Measured primary success, total tokens and elapsed time. Whiskers are descriptive 95% task-cluster intervals. All-skills loading is shown separately. Figure generated from the released data.

5.2 Selected skills versus the complete catalog

The hierarchical condition uses 4 373 tokens per run on average with one selected skill, compared with 8 518 when all ten bodies are loaded. The paired mean difference is 4 145 tokens, with a 95% interval [3898; 4366]. The increase is 94.8%, interval [87.9; 101.9] %.

Strict success changes from 20/24 to 21/24; under the wrapper sensitivity it changes from 24/24 to 24/24. The mean latency difference, all minus selected, is 0.74 seconds, interval [-0.24; 1.83]. This is a payload ablation with the correct family supplied in advance. It does not measure catalog-search errors or their token cost.

5.3 Costs and uncertainty

The reflexive workflow executes 5 revision calls across its 24 runs; a critic can add cost even when no revision follows. Provider-reported cached input totals 44 788 tokens, and reported reasoning totals 97 915. These are subsets, not additional tokens. Tables 4 and 5 retain failed attempts and reflect the small number of independent task instances. Point rankings alone should not be interpreted as statistically established general superiority.

Condition	Total tokens	Calls	Success/100k tok.	Success/min
Single agent	27 424	24	72.93	10.58
Sequential	81 845	72	24.44	3.64
Parallel	84 479	72	22.49	5.83
Hierarchical	104 963	95	19.05	4.47
Reflexive	65 133	58	30.71	5.12
Hierarchical + all skills	204 439	95	10.27	4.40

Table 4: Resource efficiency using primary strict successes and all attempts. Accumulated successes/minute is not saturated-server throughput.

Condition	Success %	Mean tokens	Mean seconds
Single agent	[62.5; 100.0]	[1005; 1279]	[3.33; 6.43]
Sequential	[58.3; 100.0]	[3094; 3752]	[10.90; 17.28]
Parallel	[54.2; 100.0]	[3157; 3904]	[5.65; 11.58]
Hierarchical	[58.3; 100.0]	[4012; 4740]	[9.04; 13.89]
Reflexive	[58.3; 100.0]	[2373; 3064]	[7.03; 12.82]
Hierarchical + all skills	[66.7; 100.0]	[7987; 9037]	[9.39; 15.22]

Table 5: Descriptive 95% task-cluster bootstrap intervals for the primary metric, mean total tokens and mean wall time. Twelve task ids are the resampling units.

6 Choosing a subagent architecture

On these short tasks, the single-agent reference spends 1 143 tokens per run, while the least token-intensive team configuration spends 2 714. Their strict completion counts must be read alongside Table 3 and the SQL sensitivity. The actionable lesson is to preserve direct execution while evaluating delegation where parallel work or context separation can repay the extra calls. This is a workload-specific recommendation, not a rejection of subagents.

Ledger	4/4	4/4	4/4	4/4	4/4	4/4
Calendar	4/4	4/4	4/4	4/4	4/4	4/4
Dependencies	3/4	4/4	3/4	4/4	4/4	4/4
Selection	4/4	4/4	4/4	4/4	4/4	4/4
Memory	4/4	4/4	4/4	4/4	4/4	4/4
SQL	1/4	0/4	0/4	0/4	0/4	1/4
	Single agent	Sequential	Parallel	Hierarchical	Reflexive	Hierarchical (all skills)

Exact successes by family: two instances × two repetitions

Figure 2: Strict successes by task family. Each cell represents four runs, not four independent task instances. The SQL sensitivity is reported separately in Table 3.

6.1 A master agent with optional delegation

An adaptive coordinator can choose between direct execution and delegation. The master interprets the request, maintains the active objective and integrates worker results. Delegation is an available operation, not a mandatory stage for every message. This conditional architecture is a recommendation derived from the trade-offs; its routing policy was not trained or benchmarked here.

For a two-worker fan-out, an approximate latency decomposition is

$$W_{\text{team}} \approx W_{\text{plan}} + \max(W_1, W_2) + W_{\text{merge}}, \quad (2)$$

whereas total tokens add across planner, both workers and merger. Concurrency only shortens the overlapping portion of the critical path. A useful delegation decision should therefore consider separable work, required handoffs and expected duration, not just the existence of multiple subtasks. Small direct actions may have too little work to amortize coordination.

An initial implementation could permit one delegation level and two concurrent workers, retaining a configurable ceiling. These values describe a manageable starting design, not an experimentally proven optimum. Increasing team size, introducing peer negotiation or mixing model sizes would require separate controlled measurements.

6.2 Skill and context assignment

The master should discover the complete metadata catalog while workers initially load the instructions relevant to their assignment. Each worker can request further permitted resources when the initial package proves insufficient. This preserves discoverability without copying every instruction into every request. The experiment’s label-based selection is easier than that real routing problem, so its token savings are not a guarantee for a deployed selector.

A practical handoff can contain `task_id`, `parent_task_id`, objective, input references, selected skill versions, allowed tools, dependencies and the expected result shape. The response should separate completed work, artifacts, unresolved facts and blockers. These fields organize execution and auditability; they do not require another model call to restate a formal proof for every action.

The experimental workers received the full original task to keep evidence access comparable. Production context scoping is a subsequent design change: supply task-specific evidence and relevant recent instructions, with retrievable references for omitted material. It must be evaluated for both reduced context and increased failures from missing information. The present results cannot establish that a smaller context is always sufficient.

6.3 Runtime responsibilities

Workers can be scheduled as asynchronous tasks sharing provider clients and storage connections. A process is needed only when an invoked tool actually requires one. This avoids equating an agent with a full operating-system process, but host memory was not measured in this study and no RAM reduction is claimed.

The scheduler should bound worker concurrency, propagate cancellation and preserve dependency order. A master waiting for children must not monopolize the only execution slot they need; orchestration and worker scheduling require compatible admission rules. Worker permissions cannot exceed the parent's grants, and conflicting writes require coordination rather than unrestricted simultaneous mutation.

The coordinator can expose delegation, status, communication and cancellation through a small runtime interface. Escalation of unresolved decisions to the coordinator preserves a clear responsibility boundary. Ordinary typed tool results and error handling remain engine responsibilities. A model critic, as evaluated in the reflexive condition, is an optional extra computation whose value must be demonstrated for the workload.

7 Limits, reproducibility and next experiment

The task set is small, synthetic and designed for this study. Short, fully supplied inputs favor direct solving and do not exercise document retrieval, terminal actions, multi-file code changes or long-running failure recovery. Only one model and reasoning setting were used, with two repetitions per instance. Provider caching, request scheduling and same-day network conditions affect measured latency. Equal maximum call limits do not produce an equal-compute comparison.

The post-hoc SQL-format sensitivity is necessary to interpret the primary errors, but it is not a second preregistered endpoint. It changes only the wrapper of an existing string and preserves the original strict scores. It cannot correct SQL logic, answer content or infrastructure failures. Its interpretation must remain separate from the frozen primary analysis.

The released bundle includes the task generators, exact prompts, ten procedural skills, evaluators, local protocol, source hashes, randomized order, visible responses, usage, timestamps, CSV/XLSX tables and both LaTeX versions. The source audit checks complete blocks and reconciles tokens per call with tokens per run. An editorial amendment register documents post-evaluation protocol prose changes without replacing historical hashes or modifying experimental settings. Credentials and private reasoning are excluded. The model alias may evolve, so the bundle reproduces the method and recorded analysis, not a deterministic remote service.

A stronger next experiment should reserve real assistant tasks with external actions, longer contexts, explicit dependencies and conflicting writes; test more models; and compare adaptive delegation with direct execution at matched budgets. It should separately vary context scoping, skill discovery and team size. This study provides a reproducible starting measurement, not evidence that one fixed team design is universally best.

8 Conclusion

Architecture selection should consider delivered correctness, total inference and actual waiting time together. The comparison makes coordination overhead visible and shows why output-format compliance must be distinguished from solution quality. For the measured short tasks, direct execution is the most economical baseline. Among the multi-agent patterns, reflexive coordination uses the fewest tokens on average and parallel coordination has the lowest mean latency. No universal winner is established; adaptive, scoped delegation remains a hypothesis to test across workloads.

Availability and status. The bilingual paper and experimental bundle are available from the CREH Labs research page. This is an exploratory preprint without peer review. The benchmark was executed in an independent research harness and evaluates bounded coordination patterns.

References

1. Siddhant Kulkarni and Yukta Kulkarni. *Benchmarking Multi-Agent LLM Architectures for Financial Document Processing: A Comparative Study of Orchestration Patterns, Cost-Accuracy Tradeoffs and Production Scaling Strategies*. 2603.22651v1. First submitted 2026-03-24. <https://arxiv.org/html/2603.22651v1>
2. Ziqi Wang, Yuhao Yang, Zhiwei Ling, Wenzhuo Qian and Hailiang Zhao. *When Does Hierarchy Help? Benchmarking Agent Coordination in Event-Driven Industrial Scheduling*. 2605.13172v1. First submitted 2026-05-13. <https://arxiv.org/html/2605.13172v1>
3. Zedong Yu et al.. *Beyond Sequential Interaction: Benchmarking Parallel Execution and Coordination for GUI Agents*. 2607.22689v1. First submitted 2026-07-17. <https://arxiv.org/html/2607.22689v1>
4. Zhenzhen Ren et al.. *OrchBench: Evaluating Multi-Agent Orchestration Plans in Isolation via Deterministic Simulation*. 2607.25656v1. First submitted 2026-07-28. <https://arxiv.org/html/2607.25656v1>
5. Abhijnan Nath, Hannah VanderHoeven and Nikhil Krishnaswamy. *CRAFT: Grounded Multi-Agent Coordination Under Partial Information*. 2603.25268v2. First submitted 2026-03-26. <https://arxiv.org/html/2603.25268v2>
6. Wasu Top Piriyaakulkij, Rachel Lawrence, Alicia Curth, Sushrut Karmalkar and Niranjani Prasad. *Sub-agents vs Agent Skills: Executing Reusable Knowledge for Long-Horizon Agentic Tasks*. 2609.09233v1. First submitted 2026-09-07. <https://arxiv.org/html/2609.09233v1>
7. Aimin Zhang, Jiajing Guo, Fuwei Jia, Chen Lv, Boyu Wang and Fangzheng Li. *EvoAgent: An Evolvable Agent Framework with Skill Learning and Multi-Agent Delegation*. 2604.20133v3. First submitted 2026-04-22. <https://arxiv.org/html/2604.20133v3>
8. Xinyu Zhang et al.. *Swarm Skills: A Portable, Self-Evolving Multi-Agent System Specification for Coordination Engineering*. 2605.10052v2. First submitted 2026-05-11. <https://arxiv.org/html/2605.10052v2>
9. Renjun Xu and Yang Yan. *Agent Skills for Large Language Models: Architecture, Acquisition, Security, and the Path Forward*. 2602.12430v4. First submitted 2026-02-12. <https://arxiv.org/html/2602.12430v4>
10. OpenAI. *GPT-5.6 Luna Model*. Accessed 2026-09-13. <https://developers.openai.com/api/docs/models/gpt-5.6-luna>